METRO TRANSIT SYSTEM | SQL ANALYTICS REPORT

Metro Transit System

SQL Analytics and Operational Performance Report

Turning a relational metro-transit database into insight on demand, routes, fares, trains, stations and passenger behaviour.

4	25	465	1,490
Tables	Stations	Passengers	Bookings

PREPARED BY

R Hemant Kumar Patra

TECHNOLOGY

SQL / MySQL

REPORT TYPE

Data analytics and database project

Portfolio-ready presentation | Evidence-led analysis | Clean relational design

1. Executive Overview

A concise view of the network, the analytical scope and the database foundation.

The Metro Transit System project models a city rail network through four connected relational tables: **Stations, Trains, Passengers and TicketBookings**. It is designed to demonstrate SQL from database constraints through to advanced analytical techniques, turning raw bookings into operational intelligence.

25	20	465	1,490
Stations	Trains	Passengers	Bookings

Project Purpose

Translate a relational metro-transit database into operational insights covering passenger demand, route utilisation, fare behaviour, train performance, station activity and SQL engineering practices.

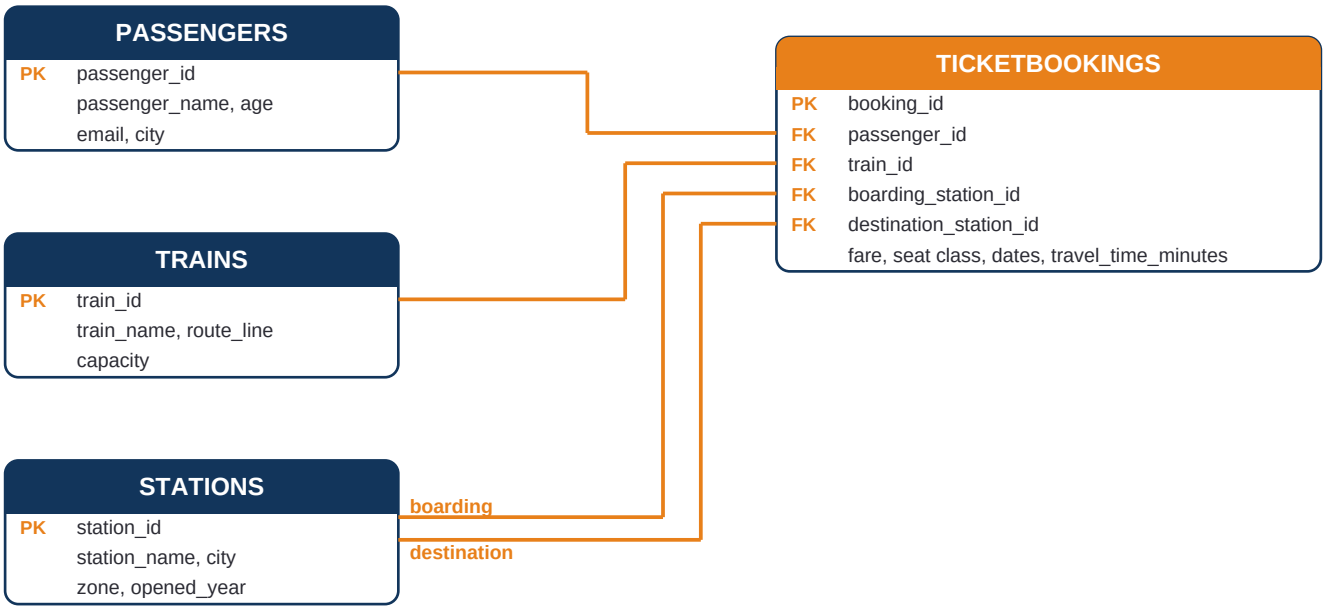
Analytical Scope

Area	Business question	SQL focus
Demand	Where is passenger activity concentrated?	GROUP BY, HAVING, JOIN
Revenue	Which routes and seat classes drive value?	SUM, AVG, CASE
Operations	Which trains and stations show stronger activity?	JOIN, aggregation
Passengers	Who are the most active and valuable users?	COUNT, subqueries
Time	How does booking activity change over time?	Date functions

Data integrity. Record counts above come from the project guide. Revenue, percentages, rankings and trend findings are calculated only after loading the complete SQL dataset, so this report states no unverified results.

2. Database Architecture

The relational structure supports both operational reporting and advanced SQL analysis.



Core Entities

Table	Role in the system	Key fields
Stations	Metro station master data	station_id, station_name, city, zone, opened_year
Trains	Train and route information	train_id, train_name, route_line, capacity
Passengers	Registered passenger information	passenger_id, passenger_name, age, email, city
TicketBookings	Transactional booking records	booking_id, passenger_id, train_id, boarding_station_id, destination_station_id, fare and travel details

Relationship Map

Parent / master	Relationship	Transaction
Passengers	passenger_id to passenger_id	TicketBookings
Trains	train_id to train_id	TicketBookings
Stations	station_id to boarding_station_id	TicketBookings
Stations	station_id to destination_station_id	TicketBookings

Design highlight. TicketBookings references Stations twice, once for the boarding station and once for the destination. This is a real-world example of multiple foreign keys pointing to the same parent table.

3. Demand and Network Utilisation

Where do passengers actually use the network?

Primary question: where is passenger demand concentrated across routes, trains and stations?

Analysis	Measure	Recommended presentation
Route-line demand	Bookings + booking share	Ranked bar / table
Train demand	Bookings + revenue	Top and low performers
Boarding activity	Boarding count + revenue	Station ranking
Destination activity	Destination count	Station ranking
Monthly demand	Bookings by month	Trend chart

Analytical Interpretation

The strongest demand story combines booking volume with revenue rather than relying on a single metric. A route can lead in bookings while another generates stronger revenue through a higher average fare.

PENDING DATA
Identify the leading route line, the most-used train and the highest-activity boarding station after running the complete SQL dataset.

4. Revenue and Fare Performance

How does booking volume translate into fare revenue?

Dimension	Core metrics	Decision value
Route line	Bookings, revenue, average fare	Compare demand with monetary contribution
Seat class	Bookings, revenue, revenue share	Understand class-level value
Fare category	Low / Medium / High	Profile fare distribution

Suggested Fare Classification

Category	Rule	Output
Low fare	Project-defined lower fare band	Booking count + share
Medium fare	Project-defined middle fare band	Booking count + share
High fare	Project-defined upper fare band	Booking count + share

Business lens. Separate high-volume segments from high-value segments. Use actual SQL results to see whether revenue is concentrated by route, seat class or fare band.

5. Passenger Behaviour and Time Analysis

From network performance to how customers use the service.

Passenger Behaviour

Metric	SQL analysis
Age profile	MIN, MAX, AVG age
Email completeness	Count missing email values
Repeat usage	Passengers with more than one booking
High-value passengers	Top passengers by total fare paid
Passenger cities	Passenger count by city

Time-Based Performance

Metric	Analysis
Monthly bookings	COUNT bookings by month
Monthly revenue	SUM fare by month
Average fare trend	AVG fare by month
Travel-time profile	Average and range of travel_time_minutes

Analytical guardrail. Do not claim seasonality from a short observation period. Describe a trend only when the available booking dates provide enough evidence.

6. SQL Engineering and Analytical Techniques

A progression from database design to advanced analytical SQL.

Capability	Techniques demonstrated
Database design	PRIMARY KEY, FOREIGN KEY, UNIQUE, DEFAULT, CHECK
Aggregation	COUNT, COUNT(DISTINCT), SUM, AVG, GROUP BY, HAVING
Text and dates	UPPER, LENGTH, MONTH, YEAR, DAYNAME, DATEDIFF, DATE_ADD
Joins	INNER JOIN, LEFT JOIN, multi-table joins
Advanced SQL	Subqueries, CTEs, Views, CASE
Window functions	RANK, DENSE_RANK, LAG, LEAD
Database operations	Transactions, ROLLBACK, indexes, composite indexes

7. Problem-Solving Patterns

Selected business questions and the SQL approach that answers each.

Business question	SQL approach
Which trains exceed a booking threshold?	JOIN + GROUP BY + HAVING
Which passengers exceed average spending?	Aggregation + subquery
How do passenger rankings change?	Window functions
How can repeated logic be simplified?	CTE / View
How can frequent searches be optimised?	Indexes / composite indexes

Illustrative Query Patterns

Written for this report to show the approach. They use only table and column names listed in the project guide; confirm names against your actual schema before running.

Trains above a booking threshold (JOIN + GROUP BY + HAVING)

```
SELECT t.train_id, t.train_name, COUNT(b.booking_id) AS total_bookings
FROM Trains t
JOIN TicketBookings b ON t.train_id = b.train_id
GROUP BY t.train_id, t.train_name
HAVING COUNT(b.booking_id) > 50;   -- threshold is an example
```

Repeat passengers (more than one booking)

```
SELECT passenger_id, COUNT(*) AS bookings
FROM TicketBookings
GROUP BY passenger_id
HAVING COUNT(*) > 1
ORDER BY bookings DESC;
```

Boarding-station activity ranking (window function)

```
SELECT s.station_name, COUNT(*) AS boardings,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS activity_rank
FROM TicketBookings b
JOIN Stations s ON b.boarding_station_id = s.station_id
GROUP BY s.station_id, s.station_name;
```

Missing passenger email (data quality)

```
SELECT COUNT(*) AS missing_emails
FROM Passengers
WHERE email IS NULL OR TRIM(email) = '';
```

Portfolio value. The project shows more than query writing: relational thinking, aggregation, comparative analysis, reusable SQL structures, window functions, transaction handling and basic query-performance awareness.

8. Findings, Recommendations and Conclusion

Translating SQL output into operational decision support.

Evidence-Led Findings

Theme	Finding to populate from the complete dataset	Status
Demand	Leading route line, train and station by booking activity	Pending
Revenue	Highest-value route and seat / fare category	Pending
Operations	Strongest and weakest train / station activity	Pending
Passengers	Repeat usage and high-value passenger behaviour	Pending
Time	Highest and lowest booking month; revenue movement	Pending

Recommended Actions

Action area	Recommendation principle
Capacity planning	Use sustained booking concentration to guide service review.
Route management	Monitor high- and low-performing lines using both bookings and revenue.
Station operations	Investigate high-demand locations and stations with weak activity.
Revenue strategy	Evaluate fare-class and route-level performance before pricing decisions.
Passenger retention	Use repeat-booking patterns to understand recurring users.
Data quality	Monitor missing passenger contact information and other completeness issues.

Conclusion

The Metro Transit System SQL project is a practical example of turning relational data into operational intelligence. By combining passengers, trains, stations and ticket transactions, the analysis can show where demand is concentrated, how revenue is generated, which parts of the network need attention and how passengers use the service over time.

Before publishing: replace every data-dependent placeholder (marked *Pending*) with verified SQL output from the complete dataset.